



React Beyond the Basics

useEffect, Data Fetching, Routing, Forms & Performance

Chapter Overview

- 01 useEffect and Side Effects
- 02 Data Fetching
- 03 React Router
- 04 Forms in React
- 05 Lifting State Up
- 06 Error Boundaries and Performance



useEffect and Side Effects

Section 1



What Are Side Effects?

Pure Rendering vs Side Effects

- Rendering is pure: given the same props and state, return the same JSX
- Side effects are anything outside pure rendering — API calls, timers, DOM changes

Examples of Side Effects

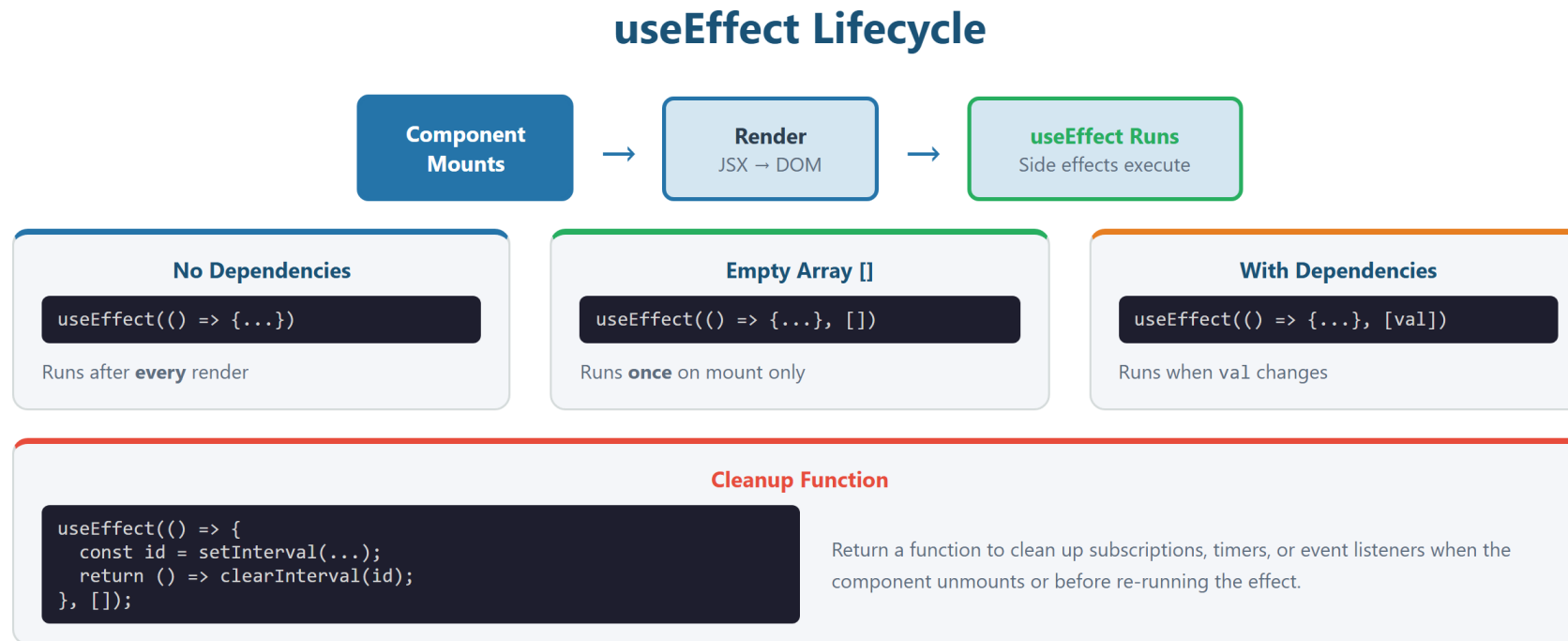
- Fetching data from an API
- Setting up a timer or interval
- Subscribing to WebSocket or browser events
- Updating document.title

Why useEffect?

- Side effects don't belong in the render phase — they run after React updates the DOM
- useEffect is React's way of saying: 'run this code after commit, generally after the browser paints'



useEffect Lifecycle



`useEffect` separates side effects from rendering — think "after the paint is done, do this"

useEffect Basics

```
import { useState, useEffect } from 'react';

function DocumentTitle() {
  const [count, setCount] = useState<number>(0);

  // Runs after every render
  useEffect(() => {
    document.title = `Count: ${count}`;
  });

  // Runs once on mount (empty dependency array)
  // Note: in dev Strict Mode, React runs an extra setup+cleanup cycle
  useEffect(() => {
    console.log('Component mounted');
  }, []);

  // Runs when count changes
  useEffect(() => {
    console.log(`Count changed to ${count}`);
  }, [count]);

  return <button onClick={() => setCount(c => c + 1)}>{count}</button>;
}
```



Cleanup Functions

Why Cleanup?

- Some effects create subscriptions, timers, or listeners that must be removed
- Without cleanup, these leak memory and cause bugs on unmount

How It Works

- Return a function from `useEffect` — React calls it before re-running the effect
- Also called when the component unmounts

Common Cleanups

- `clearInterval` / `clearTimeout` for timers
- `removeEventListener` for window/document events
- `AbortController.abort()` for cancelled fetch requests



Cleanup Example — Timer

```
function Timer() {  
  const [seconds, setSeconds] = useState<number>(0);  
  
  useEffect(() => {  
    const id = setInterval(() => {  
      setSeconds(s => s + 1); // functional update  
    }, 1000);  
  
    // Cleanup: runs on unmount  
    return () => clearInterval(id);  
  }, []); // empty deps = run once  
  
  return <p>Elapsed: {seconds}s</p>;  
}
```



Data Fetching

Section 2



Fetching Data in React

The Pattern

- Use `useEffect` with an empty dependency array to fetch on mount
- Store the result in state with `useState`
- Handle three states: loading, success, and error

The fetch API

- `fetch()` returns a Promise — use `.then()` or `async/await`
- `Response.json()` also returns a Promise — chain it
- Check `response.ok` — fetch doesn't reject on 404/500

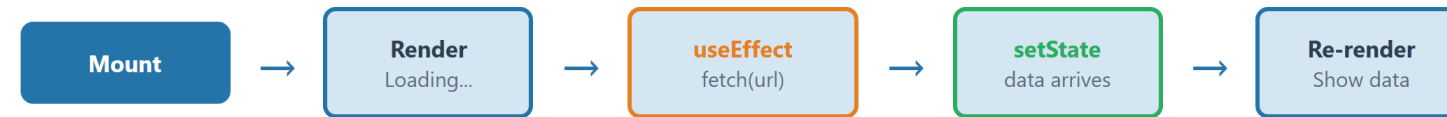
Modern Alternatives

- For many applications, prefer framework/router data APIs instead of fetching directly inside `useEffect`



Data Fetching Flow

Data Fetching with useEffect



Three State Variables

- > `data: T | null` — the fetched data
- > `loading: boolean` — show spinner?
- > `error: string | null` — error message

Pattern

```
useEffect(() => {
  fetch(url)
    .then(res => res.json())
    .then(data => { setData(data); setLoading(false); })
    .catch(err => { setError(err.message); setLoading(false); });
}, [url]);
```

Common mistake: forgetting the dependency array → infinite fetch loop

Best practice: handle loading, success, and error states for good UX

Always clean up in-flight requests if the component unmounts before the response arrives

Fetch with useEffect

```
interface User { id: number; name: string; email: string; }

function UserList() {
  const [users, setUsers] = useState<User[]>([]);
  const [loading, setLoading] = useState<boolean>(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/users')
      .then(res => {
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        return res.json();
      })
      .then((data: User[]) => { setUsers(data); setLoading(false); })
      .catch(err => { setError(err.message); setLoading(false); });
  }, []);

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error}</p>;
  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}
```



async/await in useEffect

Don't Make useEffect async Directly

- `useEffect(() => async () => {...})` is wrong — `useEffect` must return void or cleanup
- Instead, define an async function inside and call it immediately

Cancelling Requests

- Use `AbortController` to cancel fetch when the component unmounts
- Pass the signal to fetch options: `fetch(url, { signal })`
- This prevents setting state on an unmounted component



async/await with Cancellation

```
useEffect(() => {
  const controller = new AbortController();

  async function fetchData() {
    try {
      const res = await fetch(url, { signal: controller.signal });
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      const data: User[] = await res.json();
      setUsers(data);
    } catch (err) {
      if (err instanceof Error && err.name !== 'AbortError') {
        setError(err.message);
      }
    } finally {
      setLoading(false);
    }
  }

  fetchData();
  return () => controller.abort(); // cleanup
}, [url]);
```



React Router

Section 3



Client-Side Routing

The Problem

- React apps are Single Page Applications — one HTML file, one bundle
- Users still expect URLs to work: /about, /users/42, back button

React Router

- Maps URL paths to React components — no page reload
- Uses the browser's History API to change the URL
- Install: `npm install react-router-dom`

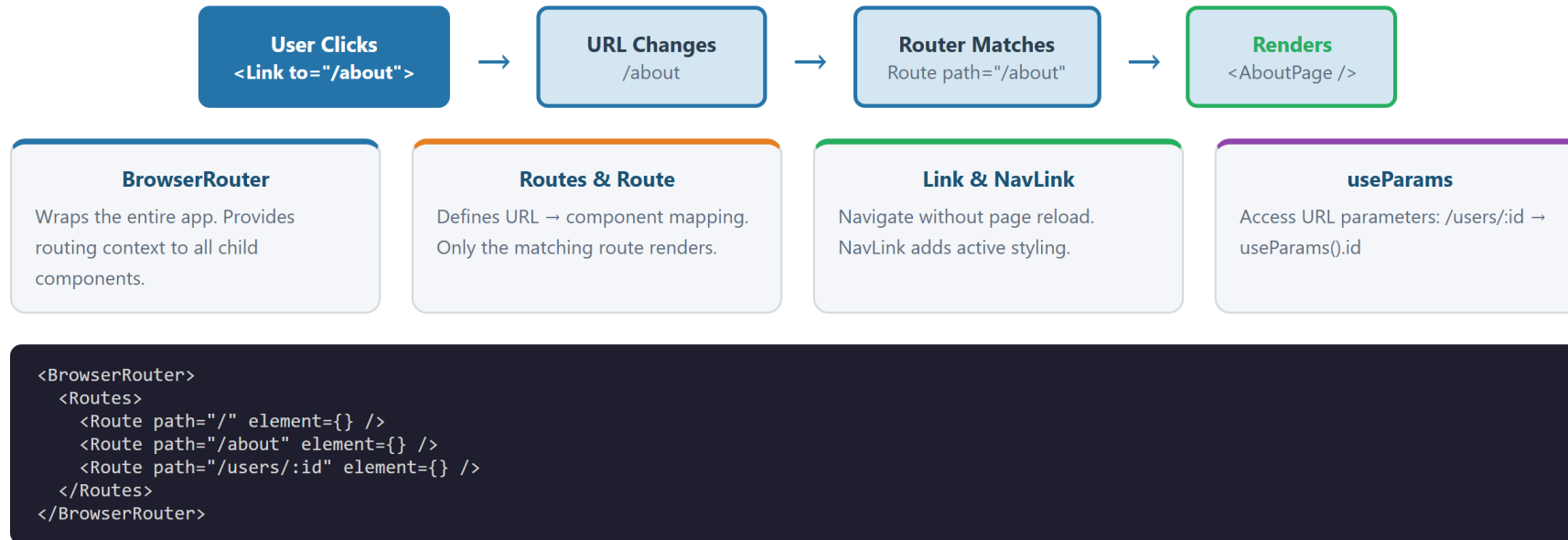
Core Concepts

- BrowserRouter wraps your app and provides routing context
- `createBrowserRouter` is recommended for data APIs and modern routing patterns
- Routes contains Route elements that map paths to components
- Link replaces `<a>` tags — navigates without page reload



React Router Flow

React Router — Client-Side Routing



No page reload — React Router swaps components based on the URL, keeping SPA feel

Setting Up Routes

```
// main.tsx
import { BrowserRouter } from 'react-router-dom';
import App from './App';

createRoot(document.getElementById('root')!).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);

// App.tsx
import { Routes, Route } from 'react-router-dom';
import Home from './pages/Home';
import About from './pages/About';
import UserDetails from './pages/UserDetail';

function App() {
  return (
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/about" element={<About />} />
      <Route path="/users/:id" element={<UserDetail />} />
    </Routes>
  );
}
```



Navigation and URL Parameters

Link and NavLink

- `<Link to='/about'>About</Link>` — renders an `<a>` tag internally
- `<NavLink>` adds an 'active' class when the URL matches — great for nav menus

URL Parameters

- Route `path='/users/:id'` — the `:id` is a dynamic segment
- `useParams()` returns `{ id: '42' }` — always strings, parse to number if needed

Programmatic Navigation

- `const navigate = useNavigate()` — call `navigate('/path')` from event handlers
- `navigate(-1)` goes back, like the browser back button



useParams and Navigation

```
import { useParams, Link, useNavigate } from 'react-router-dom';

function UserDetails() {
  const { id } = useParams<{ id: string }>();
  const navigate = useNavigate();
  const [user, setUser] = useState<User | null>(null);

  useEffect(() => {
    fetch(`https://jsonplaceholder.typicode.com/users/${id}`)
      .then(res => {
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        return res.json();
      })
      .then(data => setUser(data));
  }, [id]);

  if (!user) return <p>Loading...</p>;
  return (
    <div>
      <h1>{user.name}</h1>
      <p>{user.email}</p>
      <button onClick={() => navigate(-1)}>Back</button>
      <Link to="/">Home</Link>
    </div>
  );
}
```

Forms in React

Section 4



Controlled Components

What Is a Controlled Component?

- An input whose value is driven by React state: `value={state}`
- Every change goes through: `onChange` → `setState` → re-render → new value
- React is the 'single source of truth' for the input value

Form Elements

- `<input>` — text, number, email, password
- `<textarea>` — uses `value` prop (not `children`) in React
- `<select>` — uses `value` prop on the `<select>` element (not `selected` attribute)
- `<input type='checkbox'>` — uses `checked` prop instead of `value`

React 19 Additions

- React 19 introduces form-related hooks: `useActionState`, `useFormStatus`, and `useOptimistic`



Controlled Form

```
interface FormData {
  name: string;
  email: string;
  role: string;
  agree: boolean;
}

function SignupForm() {
  const [form, setForm] = useState<FormData>({
    name: '', email: '', role: 'developer', agree: false,
  });

  const handleChange = (e: React.ChangeEvent<HTMLInputElement | HTMLSelectElement>) => {
    const { name, value, type } = e.target;
    setForm(prev => ({
      ...prev,
      [name]: type === 'checkbox' ? (e.target as HTMLInputElement).checked : value,
    }));
  };

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    console.log(form);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name='name' value={form.name} onChange={handleChange} />
      <input name='email' value={form.email} onChange={handleChange} />
      <select name='role' value={form.role} onChange={handleChange}>
        <option value='developer'>Developer</option>
        <option value='designer'>Designer</option>
      </select>
      <label>
        <input type='checkbox' name='agree' checked={form.agree} onChange={handleChange} />
        I agree
      </label>
      <button type='submit' disabled={!form.agree}>Submit</button>
    </form>
  );
}
```



Form Validation

Client-Side Validation

- Validate on submit: check all fields before sending
- Validate on change: give instant feedback as the user types
- Store errors in state: errors: Record<string, string>

Validation Pattern

- Create a validate function that returns an errors object
- Display errors next to each field: {errors.email} && {errors.email}
- Disable submit button when errors exist



Form Validation Pattern

```
const [errors, setErrors] = useState<Record<string, string>>({});

function validate(form: FormData): Record<string, string> {
  const errs: Record<string, string> = {};
  if (!form.name.trim()) errs.name = 'Name is required';
  if (!form.email.includes('@')) errs.email = 'Invalid email';
  if (!form.agree) errs.agree = 'You must agree';
  return errs;
}

const handleSubmit = (e: React.FormEvent) => {
  e.preventDefault();
  const errs = validate(form);
  setErrors(errs);
  if (Object.keys(errs).length === 0) {
    console.log('Valid!', form);
  }
};

// In JSX:
// <input name='email' value={form.email} onChange={handleChange} />
// {errors.email && <span className='error'>{errors.email}</span>}
```



Lifting State Up

Section 5



Why Lift State?

The Problem

- Two sibling components need to share the same data
- React data flows one way: parent → child. Siblings can't share directly.

The Solution

- Move the shared state to the nearest common ancestor
- Pass the state down as props to both children
- Pass a callback to the child that needs to update the state

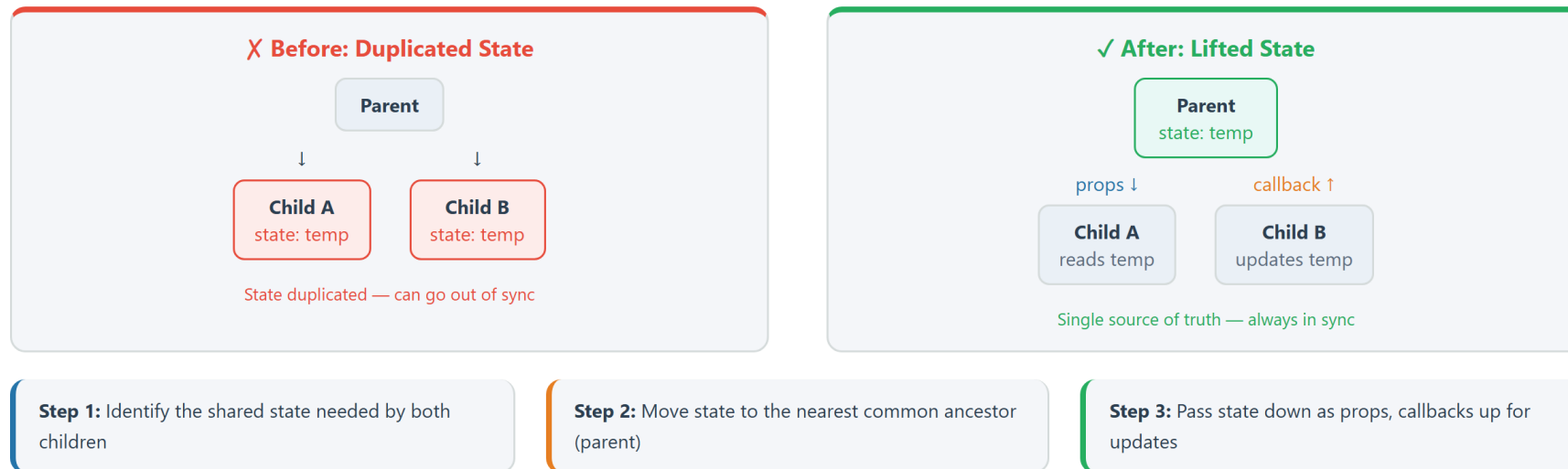
When to Lift

- When two or more components need the same changing data
- When a child's action should affect a sibling's display



Lifting State Up

Lifting State Up



When two components need the same data, lift state to their closest common parent

Lifting State Example

```
// Parent holds the shared state
function TemperatureConverter() {
  const [celsius, setCelsius] = useState<number>(0);

  return (
    <div>
      <CelsiusInput value={celsius} onChange={setCelsius} />
      <FahrenheitDisplay celsius={celsius} />
    </div>
  );
}

// Child A: updates the shared state via callback
function CelsiusInput({ value, onChange }: {
  value: number; onChange: (v: number) => void;
}) {
  return <input type='number' value={value}
    onChange={e => onChange(Number(e.target.value))} />;
}

// Child B: reads the shared state via props
function FahrenheitDisplay({ celsius }: { celsius: number }) {
  return <p>{(celsius * 9/5 + 32).toFixed(1)}°F</p>;
}
```



Error Boundaries and Performance

Section 6



Error Boundaries

What Is an Error Boundary?

- A component that catches JavaScript errors in its child tree
- Displays a fallback UI instead of crashing the entire app
- Must be a class component — hooks can't catch render errors (yet)

What It Catches

- Errors during rendering, in lifecycle methods, and in constructors
- Does NOT catch errors in: event handlers, async code, or server-side rendering

Practical Usage

- Wrap sections of your app — don't let one broken widget crash everything
- Use react-error-boundary package for a hook-friendly wrapper



Error Boundary Pattern

```
// Using react-error-boundary (npm install react-error-boundary)
import { ErrorBoundary } from 'react-error-boundary';

function ErrorFallback({ error, resetErrorBoundary }: {
  error: Error;
  resetErrorBoundary: () => void;
}) {
  return (
    <div role='alert'>
      <h2>Something went wrong</h2>
      <pre>{error.message}</pre>
      <button onClick={resetErrorBoundary}>Try again</button>
    </div>
  );
}

// Usage: wrap sections of your app
<ErrorBoundary FallbackComponent={ErrorFallback}>
  <UserProfile />
</ErrorBoundary>
```



Performance Basics

When Does React Re-render?

- When state changes — React re-renders that component and normally re-evaluates its subtree
 - Memoization such as `React.memo` can prevent some child re-renders
- When props change — the child component re-renders
- Re-renders are usually fast — React's virtual DOM diffing is efficient

React.memo

- Wraps a component to skip re-renders when props haven't changed
- Compares props shallowly — objects/arrays must be new references to trigger

useMemo and useCallback

- `useMemo` caches expensive computations: `useMemo(() => compute(data), [data])`
- `useCallback` caches functions so they don't change on every render
- Don't optimize prematurely — measure first with React DevTools Profiler



React.memo and useMemo

```
import { memo, useMemo, useCallback } from 'react';

// Memoized component – skips re-render if props are the same
const ProductCard = memo(function ProductCard({ product, onClick }: Props) {
  return (
    <div onClick={() => onClick(product.id)}>
      {product.name} – ${product.price}
    </div>
  );
});

function ProductList({ products, filter }: ListProps) {
  // Memoized computation – only recalculates when products or filter change
  const filtered = useMemo(
    () => products.filter(p => p.name.includes(filter)),
    [products, filter]
  );

  // Memoized callback – stable reference for child components
  const handleClick = useCallback(
    (id: number) => console.log('clicked', id),
    []
  );

  return filtered.map(p => (
    <ProductCard key={p.id} product={p} onClick={handleClick} />
  ));
}
```



Component Lifecycle Concepts

Mounting

- Component is created and inserted into the DOM
- `useEffect` with `[]` runs after mount — set up subscriptions, fetch data

Updating

- Component re-renders due to state or prop changes
- `useEffect` with dependencies runs after updates — react to changes

Unmounting

- Component is removed from the DOM
- `useEffect` cleanup function runs — remove listeners, cancel requests

Effects Timing

- React may run effects before paint for interaction-driven updates
- `useLayoutEffect` is required for strict before-paint logic (e.g., measuring DOM)



Key Takeaways

Key Takeaways

1. `useEffect` runs side effects after render — use the dependency array to control when
2. Data fetching follows a pattern: `useEffect` + `useState` for loading/data/error states
3. React Router maps URLs to components — no page reload, full SPA navigation
4. Controlled forms store input values in React state — single source of truth
5. Lift state to the nearest common ancestor when siblings need shared data
6. Don't optimize early — React is fast by default; profile before using `memo`/`useMemo`

